APMA4302 HW4

Jin Hu

May 9, 2026

1. (10 pts) Rewrite the governing equations in stream-function/vorticity form.

Solution:

In two dimensions, let the velocity field be

$$\mathbf{v} = (u, w).$$

The stream function is defined by

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}).$$

Therefore,

$$\mathbf{v} = \left(\frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x} \right).$$

Then the incompressibility condition is automatically satisfied:

$$\nabla \cdot \mathbf{v} = \frac{\partial}{\partial x} \left(\frac{\partial \psi}{\partial y} \right) + \frac{\partial}{\partial y} \left(-\frac{\partial \psi}{\partial x} \right) = \psi_{xy} - \psi_{xy} = 0.$$

The vorticity is defined by

$$\omega \hat{\mathbf{k}} = \nabla \times \mathbf{v}.$$

So

$$\omega = \frac{\partial w}{\partial x} - \frac{\partial u}{\partial y}.$$

Using

$$u = \psi_y, \qquad w = -\psi_x,$$

we get

$$\omega = \frac{\partial}{\partial x}(-\psi_x) - \frac{\partial}{\partial y}(\psi_y) = -\psi_{xx} - \psi_{yy} = -\nabla^2 \psi.$$

Thus,

$$-\nabla^2 \psi = \omega.$$

The temperature equation stays the same, so

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T.$$

Now we take the curl of the Stokes equation

$$-\nabla^2 \mathbf{v} + \nabla p = T \hat{\mathbf{j}}.$$

Since the curl of a gradient is zero,

$$\nabla \times \nabla p = 0.$$

Therefore, the pressure term disappears. The left side becomes

$$\nabla \times (-\nabla^2 \mathbf{v}) = -\nabla^2 (\nabla \times \mathbf{v}) = -\nabla^2 (\omega \hat{\mathbf{k}}).$$

The right side is

$$\nabla \times (T\hat{\mathbf{j}}) = \nabla \times (0, T, 0) = \frac{\partial T}{\partial x}\hat{\mathbf{k}}.$$

So we obtain

$$-\nabla^2 \omega = \frac{\partial T}{\partial x}.$$

Therefore, the system can be written as

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra}\nabla^2 T,$$

$$-\nabla^2 \omega = \frac{\partial T}{\partial x},$$

$$-\nabla^2 \psi = \omega.$$

2. (10 pts) Explore fieldsplit preconditioners for solving the biharmonic equation in PETSc and Firedrake.

Solution:

The timing information was obtained using

```
./biharm -options_file <file> -log_view | grep SNESSolve
```

for the PETSc code and

```
python3 biharm.py -log_view | grep SNESSolve
```

for the Firedrake code.

(a) PETSc C Code Comparison

The PETSc C implementation was tested with three different solver option files.

Solver Configuration	SNESSolve Time (s)
direct	1.9684
split_direct	2.2529
split_mg	0.6475

From the results, the multigrid fieldsplit solver (`split_mg`) was the fastest configuration. The direct solver and split direct solver were slower because sparse LU factorization is computationally expensive. The multigrid preconditioner reduced the computational cost significantly and converged more efficiently.

The `split_direct` method was slightly slower than the monolithic direct solve because of the additional overhead introduced by the fieldsplit decomposition.

(b) Firedrake Code Comparison

The same types of solver configurations were tested in the Firedrake finite element implementation.

Solver Configuration	SNESSolve Time (s)
monolithic direct	4.0255
fieldsplit mg	4.1121
fieldsplit direct	3.2908

In the Firedrake implementation, the fieldsplit direct solver was the fastest among the three tested configurations. The fieldsplit multigrid method was slightly slower, likely because the problem size was not large enough for multigrid

to fully outperform direct methods. The setup cost of the multigrid hierarchy may also contribute to the increased runtime.

Compared with the PETSc C code, all Firedrake runtimes were larger.

(c) Comparison Between PETSc and Firedrake

Comparing the two implementations, the PETSc C code was significantly faster than the Firedrake code. For example,

$$0.6475s(PETScsplit_mg)$$

was much faster than

$$4.1121s(Firedrakefieldsplitmg).$$

The main differences in runtime come from several factors:

- The PETSc C code uses finite differences on a DMDA grid, which is relatively lightweight and low-level.
- Firedrake uses finite element methods, which require additional assembly operations for variational forms and matrices.
- Firedrake introduces additional Python overhead and symbolic UFL form compilation.
- PETSc in C has more direct memory access and less abstraction overhead.

However, Firedrake provides a much more flexible and higher-level framework for defining PDEs and finite element discretizations. While it is slower for this example, it is generally easier to extend to more complicated multiphysics problems and higher-order finite element methods.

Overall, the experiments show that multigrid methods can provide significant performance improvements, especially in lower-level PETSc implementations.

3. (5 pts) Write a new Firedrake code based on `biharm.py` to solve the biharmonic equation.

Solution:

For this problem, I modified the Firedrake code `biharm.py` to solve the biharmonic equation with the right-hand side

$$f(x, y) = \frac{\partial T}{\partial x},$$

where

$$T(x, y) = (1 - y) + A\cos(\pi x), \qquad A = 0.1.$$

Therefore,

$$f(x, y) = -A\pi \sin(\pi x).$$

In the code, I defined the temperature field as a Firedrake function and used

```
T.dx(0)
```

to compute the symbolic derivative with respect to x. The computed streamfunction was then used to define the velocity field

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}).$$

Figure 1 shows the result. The color background shows the temperature field, the white contour lines show the streamfunction, and the arrows show the velocity field. The modified Firedrake source code is submitted separately.

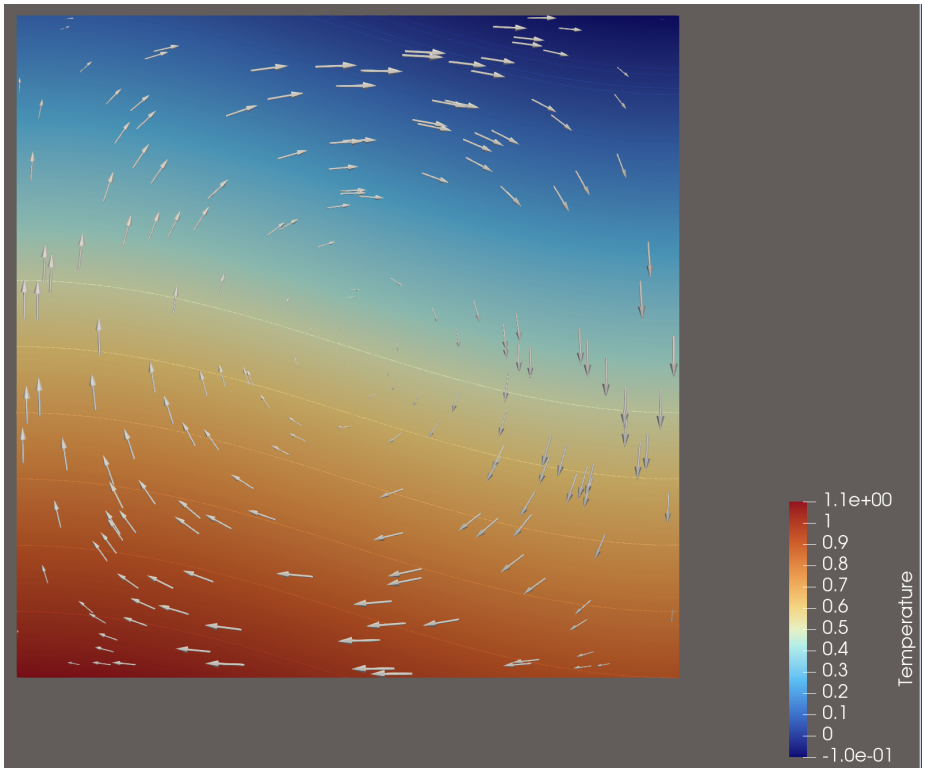

Figure 1: Temperature field with streamfunction contours and velocity vectors.

4. (20 pts) Write a Firedrake program `convection.py` that solves the full system of equations as an implicit DAE problem using `firedrake-ts`.

Solution:

(a)

For $Ra = 10^2$, I used a 64×64 Q1 mesh with an implicit BDF-2 time discretization and a monolithic direct solver using MUMPS. The final result gave

$$Nu = 1.000000.$$

The computed vorticity and streamfunction norms were also very small,

$$\|\omega\| \approx 3.77 \times 10^{-9}, \qquad \|\psi\| \approx 1.91 \times 10^{-10}.$$

This shows that the flow is essentially zero and the solution remains in the conductive state. Therefore, for $Ra = 10^2$, the steady state is non-convective.

(b)

I repeated the calculation for $Ra = 10^4, 10^5$, and 10^6 on a 64×64 Q1 mesh. The final Nusselt numbers are shown below.

Rayleigh number	Final Nusselt number Nu
10^4	4.806813
10^5	9.855926
10^6	29.121717

The results show that the Nusselt number increases as the Rayleigh number increases. This is expected because larger Ra means stronger buoyancy forcing relative to diffusion, so the heat transport becomes more convective. For $Ra = 10^4$, the solution already develops a convective state, with $Nu > 1$. For $Ra = 10^5$ and $Ra = 10^6$, the Nusselt number becomes much larger, showing stronger convective heat transport.

(c)

For $Ra = 10^4$, I computed the Nusselt number on four different $N \times N$ meshes. The results are shown below.

Mesh size	Computed Nu	Error from benchmark $Nu = 4.884$
16×16	4.526485	7.32%
32×32	4.745926	2.83%
64×64	4.806813	1.58%
128×128	4.822451	1.26%

The computed Nusselt number increases as the mesh is refined and moves closer to the Blankenbach benchmark value $Nu = 4.884$. The coarsest mesh, 16×16, underestimates the heat transport because it cannot fully resolve the thermal and velocity boundary layers. As the mesh is refined to 64×64 and 128×128, the result becomes much closer to the benchmark value. The remaining difference is likely due to the finite final time, the time step size, and the fact that I used a hand-implemented implicit BDF-2 update instead of the original `firedrake-ts` integrator.